

Security & Observability Design - Detailed

PART A - SECURITY

1. Security Objectives

Objective	Meaning for SALESTORM
Confidentiality	Customer PII and payment tokens are protected in transit and at rest
Integrity	Prices, stock, payment and order states cannot be tampered with
Availability	Abuse and DDoS cannot starve genuine customers during the sale
Accountability	Every sensitive action is attributable and auditable
Fairness	Bots and multi-account abuse cannot hoard the 100 units

2. Trust Boundaries

Internet (untrusted) → CDN/WAF → LB → API Gateway (trust boundary 1)
API Gateway → internal services (mTLS; trust boundary 2)
Services → databases / Kafka / Redis (private subnets; trust boundary 3)
Services ↔ payment gateway, couriers, SMS/email (external; trust boundary 4)

3. Authentication

Topic	Design
Customers	OAuth2 / OIDC through the Identity Provider; password + optional OTP; short-lived JWT access token (15 min) + rotating refresh token
Admin / Support	SSO + mandatory MFA; separate admin realm; IP allow-list for admin APIs
Service-to-service	mTLS with short-lived certificates from the service mesh; service identity (SPIFFE style)
Webhooks	HMAC-SHA256 signature over raw body with a shared secret, timestamp tolerance ±5 min, event-id replay protection
Sessions	HttpOnly, Secure, SameSite cookies; token revocation list for logout/compromise
Pre-sale	Users must be logged in and verified (email/phone) before the sale starts; accounts created after announcement are throttled

4. Authorization (RBAC + ownership)

Role	Permissions
customer	Browse; manage own cart; create/read/cancel own reservation, payment, order
support	Read any order/payment; initiate refunds; no config changes
admin	Configure sale, load stock, pause/resume sale, trigger reconciliation
service	Narrow scopes per service (e.g. Order Service may read payment status, not create payments)

Enforcement: coarse role checks at the gateway; **object-level ownership check in every service** (`reservation.customer_id == token.sub`) to prevent IDOR (Insecure Direct Object Reference). Deny by default.

5. Transport and Network Security

Control	Detail
HTTPS	TLS 1.2+ (1.3 preferred), HSTS, modern cipher suites, certificate automation
Internal	mTLS between services; Kafka TLS + SASL, topic ACLs per service
Network	Private subnets for services, DB, Redis, Kafka; security groups allow only required ports; no public DB endpoints
Egress	Egress allow-list to payment gateway/couriers only
WAF	OWASP Top 10 rule set, bot management, geo/IP reputation, rate rules
DDoS	CDN-level volumetric protection, SYN-flood protection at LB

6. Rate Limiting and Abuse Protection (flash-sale specific)

Threat	Control
Bots hoarding stock	Bot score at WAF; CAPTCHA / proof-of-work challenge at Buy Now; device fingerprint signals; verified accounts only
Multi-account abuse	Limit 1 unit per customer and per verified phone/payment instrument/address (policy configurable); velocity checks
Request floods	Per-IP and per-user token buckets (e.g. 5 buy requests/s/user); global admission token bucket
Speed advantage via scripts	Admission tokens bound to user+sale; randomised queue after sale opens

Stock hoarding by unpaid reservations	Short TTL, payment intent required quickly, per-user limit, release on timeout
Replay of requests	Idempotency-Key + admission token single-use + timestamp checks
Enumeration / scraping	Rate limits, opaque UUIDs (no sequential ids), response normalisation

7. Input Validation

- JSON-schema validation at the gateway (types, ranges, max sizes, unknown fields rejected).
- Business validation in services (quantity ≥ 1 and $\leq$ limit; sale window; ownership).
- Parameterised SQL only; ORM/query builders; no string-built queries (SQL injection).
- Output encoding in UI (XSS); strict Content-Security-Policy .
- Server-side recomputation of price and totals; the client never supplies the amount as authority.
- Request size/time limits; reject oversized or slow requests (slowloris).

8. Payment Data Security (PCI-DSS scope minimisation)

Control	Detail
No card data on our servers	Gateway-hosted fields / redirect / SDK tokenisation → SAQ-A style scope
Stored data	Only token, last 4 digits, brand, merchant_ref , gateway_txn_id
Webhook integrity	HMAC verification, replay protection, response only after durable write
Amount integrity	Server compares gateway-reported amount with expected amount; mismatch → flag + hold
Refunds	Only via Payment Service, role-checked, audited
Logging	Never log PAN/CVV/tokens; mask identifiers
Idempotency	Prevents double charge from retries (see ADR-007/008)

9. Data Protection and Secrets

Topic	Design
Encryption at rest	DB, backups, Kafka disks, object storage encrypted (KMS-managed keys)
Field-level encryption	Phone, address, email for high-sensitivity stores; envelope encryption
Data minimisation	Collect only what fulfilment needs; retention schedule; delete/anonymise on request
Secrets	Vault/KMS; short-lived dynamic DB credentials; rotation ≤ 90 days; no secrets in code/images/env dumps
Backups	Encrypted, access-controlled, restore-tested
Keys	Separate keys per environment and per data class; key rotation policy

10. Audit Logging

Aspect	Design
What	Login/logout, reservation create/release, payment initiate/success/fail/refund, order state changes, admin actions (sale open/pause, stock load, reconciliation, manual refund), permission denials
Fields	timestamp, actor_id, actor_role, action, entity, entity_id, before/after (hash or diff), result, ip, user_agent, trace_id
Storage	Append-only audit_log table + immutable object storage (WORM), shipped asynchronously
Integrity	Hash chaining or signed batches to detect tampering
Retention	1 year hot searchable, 7 years archive for financial records
Access	Read-only for auditors; separate from application admins

11. Threat Model (STRIDE)

Category	Threat	Example	Mitigation
Spoofing	Forged identity / webhook	Fake "payment succeeded" call	JWT validation, mTLS, webhook HMAC + replay check, query gateway for confirmation on suspicious events
Tampering	Change price/qty, modify stock	Edited request body	Server-side pricing, signed admission token, DB constraints, parameterised queries
Repudiation	"I never bought that"	Dispute	Audit log with trace ids, order status history, gateway references
Information disclosure	PII leak, IDOR	Reading others' orders	Ownership checks, field encryption, log masking, minimal responses
Denial of service	Flood at sale start	Bot swarm	CDN/WAF, rate limits, waiting room, load shedding
Elevation of privilege	Customer calls admin API	Token scope abuse	RBAC, separate admin realm, network allow-list, least-privilege service scopes
Business logic	Oversell, double spend, stock hoarding	Duplicate clicks, retries	Atomic UPDATE, idempotency, TTL, per-user limit
Supply chain	Vulnerable dependency	Compromised library	SBOM, dependency scanning, signed images, pinned versions

12. Secure Development and Operations

SAST/DAST in CI, dependency scanning, container image scanning, least-privilege IAM, pre-sale penetration test and load test, security incident runbook, change freeze during the sale, dedicated on-call security contact.

PART B - OBSERVABILITY

13. Observability Goals

Answer in under one minute: *Is the sale healthy? Is stock correct? Are payments flowing? Are paid customers getting orders?* Detect inventory inconsistency, payment failure and queue backlog **before** customers do.

14. Metrics

14.1 Golden signals per service

Signal	Metric (examples)
Rate	http_requests_total{service,route,status}
Errors	http_errors_total, grpc_failures_total
Latency	http_request_duration_seconds histogram (p50/p95/p99)
Saturation	CPU, memory, thread pool, DB pool wait, Kafka lag

14.2 Business metrics

Domain	Metrics
Traffic / queue	waiting_room_queue_length, admitted_per_second, rate_limited_total, bot_blocked_total
Inventory	inventory_available{product}, inventory_reserved, inventory_sold, reservation_attempts_total, reservation_success_total, reservation_failed_total{reason=OUT OF STOCK\ LIMIT\ ERROR}, reservation_expired_total, reservation_released_total{reason}, inventory_invariant_violations_total, redis_db_stock_drift
Payment	payment_initiated_total, payment_succeeded_total, payment_failed_total{code}, payment_timeout_total, payment_pending_age_seconds, gateway_latency_seconds, circuit_breaker_state{provider}, webhook_lag_seconds, refund_total
Order	orders_created_total, order_state_total{state}, payments_without_order_age_seconds, order_creation_latency_seconds
Messaging	kafka_consumer_lag{group}, dlq_depth{topic}, outbox_unpublished_age_seconds, outbox_unpublished_count
Funnel	funnel_viewed, funnel_admitted, funnel_reserved, funnel_payment_started, funnel_paid, funnel_order_confirmed, funnel_delivered

14.3 Order conversion

conversion = orders_confirmed / reservations_created (expected ≈ 95% for this test case); also reserve-pay and pay-order ratios to localise drop-offs.

15. Structured Logging

15.1 Standard fields (JSON, one line per event)

ts, level, service, env, version, event, traceId, spanId, requestId, customerIdHash, reservationId, paymentId, orderId, productId, idempotencyKey, result, latency_ms, error_code

15.2 Critical business events

Event name	When	Key fields
reservation.created / .rejected / .replayed	Buy Now outcome	reservationId, qty, available_after, reason
reservation.expired / .released / .confirmed / .sold	Lifecycle	reason, state_before/after
payment.initiated / .succeeded / .failed / .timeout / .reconciled / .refunded	Payment lifecycle	merchantRef, gatewayTxnId, amount, attempt_no
order.created / .state_changed / .creation_failed	Order lifecycle	from/to, reason
outbox.published, dlq.enqueue, consumer.duplicate_skipped	Messaging	eventId, topic, attempts
invariant.violation	Reconciler detects error	expected, actual, productId
admin.action	Sale control	actor, action

15.3 Sample

```
{
  "ts": "2026-10-05T11:03:12.482Z",
  "level": "INFO",
  "service": "inventory",
  "event": "reservation.created",
  "traceId": "4bf92f3577b34da6a3ce929d0e0e4736",
  "reservationId": "R1",
  "productId": "X",
  "customerIdHash": "9f2c...",
  "idempotencyKey": "K1",
  "qty": 1,
  "available_after": 99,
  "latency_ms": 14
}
```

Rules: no PII or card data; correlation id on every line; sampled debug during the sale; retention 30 days hot, 1 year cold, audit 7 years; log levels enforced; central aggregation (Loki/ELK).

16. Distributed Tracing

Aspect	Design
Standard	OpenTelemetry; W3C <code>traceparent</code> header
Propagation	Gateway → services (HTTP/gRPC headers) → Kafka message headers → consumers (async leg stays in the same trace)
Span attributes	<code>reservation.id</code> , <code>payment.id</code> , <code>order.id</code> , <code>product.id</code> , <code>idempotency.key(hash)</code> , <code>outcome</code>
Sampling	100% for errors and for the buy path past the gate (low volume); 1-5% for browse; tail-based sampling for slow traces
Trace shape	See below

```
POST /checkout
├ gateway: auth, rate-limit
├ checkout.price (strategy)
├ inventory.validateReservation
├ payment.initiate ─ gateway.charge (circuit breaker)
└ (async) payment.webhook → outbox → kafka → order.create → inventory.markSold → shipment.create → notification.send
```

Use cases: find where a slow checkout spent time; prove a paid customer got an order (trace from webhook to order); debug the 30 s Order outage backlog.

17. Dashboards

Dashboard	Panels
Sale war-room	Requests/s, queue length, admitted/s, stock gauges (Redis vs DB), reservations by state, conversion funnel, latency p99
Payments	Success/failure/timeout ratio, gateway latency, breaker state, pending reconcile count, refunds
Order pipeline	Orders created/min, payment-without-order age, Kafka lag, DLQ depth, outbox age
Infrastructure	Pods, CPU/memory, DB connections/replication lag, Redis latency, Kafka brokers
Security	WAF blocks, bot score distribution, 401/403/429 rates, admin actions

18. Alerts

Alert	Condition (example)	Severity	Immediate action
Inventory inconsistency	<code>inventory_invariant_violations_total > 0</code> OR <code>sold > total</code> OR <code>available < 0</code>	P1 page	Pause sale, run reconciliation, investigate
Redis/DB drift	<code>abs(redis_stock - db_available) > 2</code> for 30 s	P2	Resync from DB
Reserve latency	<code>p99 > 1 s</code> for 2 min	P2	Scale, check DB/Redis
5xx rate	<code>> 1%</code> for 2 min	P2	Inspect logs/traces
Payment failure spike	failure ratio <code>> 10%</code> for 5 min (baseline 5%)	P1/P2	Check gateway, failover provider
Breaker open	<code>circuit_breaker_state == OPEN</code> <code>> 1 min</code>	P1	Failover, TTL grace
Pending payments aging	<code>payment_pending_age_seconds > 120</code>	P2	Run reconciliation
Queue backlog	Kafka lag <code>> 30 s</code> or waiting-room length growing with no admits	P2	Scale consumers/admission
DLQ not empty	<code>dlq_depth > 0</code>	P2	Inspect, replay
Payment without order	<code>payments_without_order_age_seconds > 120</code>	P2	Run order recovery
Outbox stuck	<code>outbox_unpublished_age_seconds > 10</code>	P2	Check relay/Kafka
DB replication lag	<code>> 1 s</code>	P1	Verify standby
Security	WAF block spike, auth failures spike, admin login anomaly	P2	Security on-call

Alerting principles: alert on symptoms and invariants, not every cause; page only actionable P1s; every alert links to a runbook and the relevant dashboard.

19. SLIs / SLOs

SLI	SLO
Reserve availability	99.95%
Reserve latency p99	< 800 ms
Paid order created within 60 s of payment	99.9%
Oversell incidents	0
Duplicate payment incidents	0
Notification delivered within 5 min	99%
Error budget policy: burning > 50% of the monthly budget freezes non-essential deployments.	

20. Observability for the Failure Scenarios

Scenario	What the operator sees

Order Service down 30 s	Kafka lag and <code>payments_without_order_age</code> rising; alert; after recovery lag returns to zero and orders appear
Payment gateway outage	Breaker OPEN, pending count rising, failure ratio spike
DB failover	Connection errors, replication alerts, brief 503s, then recovery; reconciliation findings
Duplicate storm	<code>reservation.replayed</code> rate up, stock unchanged
Stock reaches zero	<code>StockExhausted</code> event, gate counter 0, <code>OUT_OF_STOCK</code> dominates

21. Jury Q&A

Q: How do you know inventory is correct right now? Gauges `available/reserved/sold` plus the invariant check every few seconds; any violation pages immediately. **Q: How would you debug a customer who paid but has no order?** Search by `paymentId` → trace shows webhook, outbox, Kafka, order consumer; DLQ or recovery job state; replay safely thanks to idempotency. **Q: How do you stop bots?** Layered: WAF/bot scoring, verified accounts, per-user limits, CAPTCHA/proof-of-work, randomised queue, one unit per customer/instrument. **Q: What about card data?** It never reaches our servers; we store tokens only.